МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ ГОМЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ П. О. СУХОГО

Факультет автоматизированных и информационных систем

Кафедра «Информатика»

Специальность 1-40 04 01 «Информатика и технологии программирования»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к дипломному проекту

на тему: «**Web-приложение онлайн-голосования в системе образования**»

Исполнитель: студент гр. ИП-41 Загорцев В. В.

Руководитель: Прокопенко Д. В.

Дата проверки: _____

Дата допуска к защите: _____

Дата защиты: _____

Оценка работы: _____

Подписи членов комиссии

По защите дипломной работы: _____

Гомель 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ 4
1.1 Анализ предметной области 4
1.2 Аналитический обзор существующих аналогов 5
1.3 Обзор технологий для реализации программного обеспечения 7
1.4 Анализ инструментальных средств автоматизации разработки и тестирования 9
1.5 Постановка задачи на дипломное проектирование 11
2 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ 13
2.1 Общая структура приложения 13
2.2 Функциональная модель программного комплекса 13
2.3 Информационная модель программного комплекса 13

ВВЕДЕНИЕ

Современные образовательные учреждения активно внедряют информационные технологии для автоматизации различных процессов управления и взаимодействия между участниками образовательного процесса. Одним из таких процессов является проведение опросов среди студентов, преподавателей и администрации учреждения.

Традиционные методы проведения голосования, такие как бумажные анкеты или устные опросы, обладают рядом недостатков: значительными временными затратами, сложностью обработки результатов, возможностью ошибок при подсчёте голосов и недостаточной прозрачностью процесса. В связи с этим возникает необходимость создания автоматизированной системы онлайн-голосования, обеспечивающей удобство проведения голосований и достоверность результатов.

Одним из перспективных решений является использование технологии блокчейн. Она позволяет создавать системы голосования, в которых каждый голос фиксируется и не может быть изменён после записи, а также обеспечивается возможность независимой проверки корректности подсчёта голосов каждым участником.

Целью данной дипломной работы является разработка веб-приложения онлайн-голосования для системы образования на основе технологии блокчейн, обеспечивающего удобство использования, прозрачность и надежность хранения данных.

В рамках дипломной работы будет разработана информационная модель платформы, включающая основные сущности системы: пользователей, голосования, варианты выбора. Также в системе будет реализована новостная лента для публикации актуальной информации для пользователей.

Результатом работы станет программный комплекс платформы онлайн-голосования, который может применяться в образовательных учреждениях для опросов студентов и преподавателей, оценки качества образовательного процесса, а также для организации голосований при проведении конференций, конкурсов, олимпиад и других образовательных мероприятий.

Задачами дипломной работы являются:

– изучение особенностей разработки клиент-серверных архитектур;

– проведение анализа технологии блокчейн и возможностей её применения в сфере онлайн-голосований;

– разработка архитектуры *Web*-приложения для проведения для проведения голосований с учётом различных образовательных сценариев;

– проектирование структуры базы данных для хранения необходимой информации;

– реализация программных модулей приложения, включая пользовательский интерфейс, авторизацию и аутентификацию пользователей, интеграцию со смарт-контрактами;

– проведение тестирования разработанного программного обеспечения.

АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Анализ предметной области

В образовательных учреждениях регулярно возникает необходимость принятия коллективных решений, связанных с организацией учебного процесса, деятельностью студенческого самоуправления и проведением различных мероприятий. Одним из основных механизмов принятия таких решений является голосование. Голосование позволяет учитывать мнение участников образовательного процесса, включая студентов, преподавателей и администрацию, и принимать решения на основе большинства голосов.

В системе образования голосования могут использоваться в различных ситуациях. Например, они применяются при выборе представителей студенческого самоуправления, проведении опросов среди студентов, выборе тем проектов, а также при проведении конкурсов, хакатонов и научных конференций. В таких случаях важно обеспечить удобство участия пользователей и корректность подсчёта результатов.

Традиционные способы проведения голосований в образовательных учреждениях, такие как бумажные анкеты или очные опросы, обладают рядом недостатков. К ним относятся значительные временные затраты на организацию и проведение голосования, сложность обработки результатов, возможность ошибок при подсчёте голосов, а также ограниченная доступность для участников, находящихся вне учебного учреждения.

С развитием информационных технологий широкое распространение получили системы электронного голосования. Электронное голосование представляет собой процесс подачи и подсчёта голосов с использованием информационных систем и сетевых технологий. Использование таких систем позволяет автоматизировать процедуру голосования, ускорить обработку результатов и обеспечить возможность участия пользователей независимо от их местоположения.

Однако большинство существующих систем электронного голосования основаны на централизованной архитектуре. Это означает, что все данные о голосовании, включая информацию о пользователях и их голосах, хранятся на серверах одной организации или оператора системы. Такой подход может вызывать вопросы, связанные с безопасностью хранения данных, прозрачностью процесса голосования и уровнем доверия со стороны участников.

Одной из основных проблем централизованных систем является возможность изменения или удаления данных при наличии доступа к серверной инфраструктуре. Пользователи не всегда имеют возможность убедиться в том, что их голос был корректно зафиксирован и учтён при подсчёте результатов. В условиях, когда важна прозрачность процесса принятия решений, такие ограничения могут снижать доверие к системе голосования.

В связи с перечисленными проблемами возрастает интерес к использованию децентрализованных технологий для реализации систем голосования. Одним из наиболее перспективных решений является применение технологии блокчейн. Блокчейн представляет собой распределённую систему хранения данных, в которой информация

записывается в виде последовательности блоков и защищается с использованием криптографических методов. Основным преимуществом данной технологии является невозможность изменения ранее записанных данных без согласия участников сети.

Использование блокчейн-технологии в системах онлайн-голосования позволяет повысить прозрачность и надёжность процесса голосования. Каждый голос может фиксироваться в системе в виде криптографической записи, что обеспечивает неизменяемость данных и позволяет участникам убедиться в корректности учёта их голосов. Кроме того, применение смарт-контрактов позволяет автоматизировать процесс регистрации голосов и подсчёта результатов.

Таким образом, анализ предметной области показывает, что существующие системы электронного голосования обладают рядом ограничений, связанных с централизованным хранением данных и недостаточной прозрачностью процедур. Использование современных веб-технологий в сочетании с блокчейн-технологией позволяет создать более надёжную и прозрачную систему онлайн-голосования, обеспечивающую удобство использования и возможность независимой проверки результатов.

1.2 Аналитический обзор существующих аналогов

С развитием цифровых технологий появилось большое количество систем электронного голосования, предназначенных для проведения опросов, выборов и принятия коллективных решений. Такие платформы позволяют автоматизировать процесс голосования, упростить участие пользователей и ускорить обработку результатов. Однако большинство существующих решений имеют централизованную архитектуру и обладают рядом ограничений, связанных с безопасностью, прозрачностью и возможностью независимой проверки результатов.

Одной из наиболее известных платформ онлайн-опросов является система *SurveyMonkey*. На рисунке 1.1 представлен интерфейс выбора цели использования данной системы.

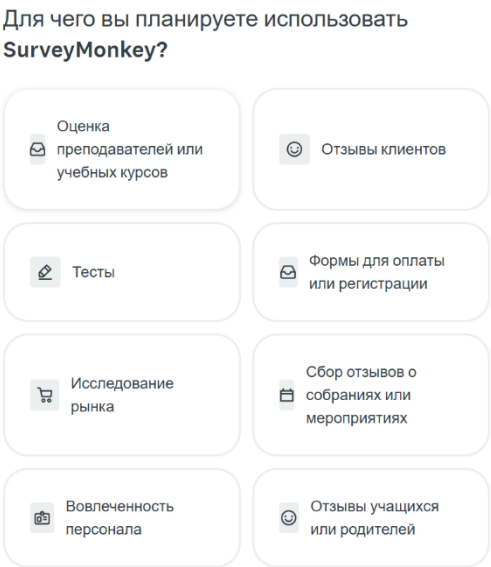

Рисунок 1.1 – Интерфейс выбора цели использования системы *SurveyMonkey*

После выбора цели пользователь получает доступ к основному функционалу системы, включающему инструменты для создания анкет, проведения опросов и анализа полученных результатов. Рассмотрим процесс создания вопроса опроса, где пользователь может задать текст вопроса, выбрать тип ответа (например, множественный выбор), добавить варианты ответов для участников опроса, а также настраивать структуру и логику опроса. Данный интерфейс представлен на рисунке 1.2.

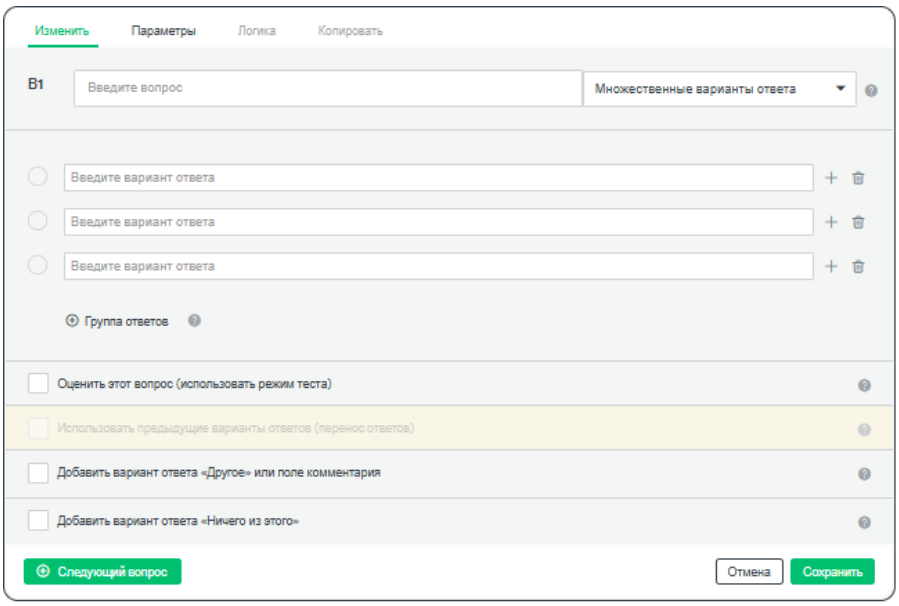

Рисунок 1.2 – Интерфейс создания вопроса в системе *SurveyMonkey*

Несмотря на удобство интерфейса и широкий набор функций, система *SurveyMonkey* является централизованной платформой. Все данные опросов и результаты голосования хранятся на серверах компании, а участники опроса не имеют возможности независимо проверить неизменность результатов голосования.

Другим распространённым инструментом является *Google Forms*. Данный сервис позволяет быстро создавать формы для опросов и голосований, распространять их среди участников и автоматически собирать результаты. Одним из основных преимуществ платформы является простота использования и интеграция с другими сервисами экосистемы *Google*. Однако, как и в предыдущем случае, система не обеспечивает полной прозрачности процесса голосования, поскольку результаты полностью контролируются владельцем формы.

Отдельную категорию составляют специализированные системы электронного голосования. Примером такой системы является *Helios Voting*. Данная платформа предназначена для проведения защищённых онлайн-выборов и использует криптографические методы для обеспечения анонимности и проверки голосов. Система предоставляет возможность независимой проверки результатов голосования, что повышает доверие участников. Тем не менее управление системой осуществляется централизованно, а хранение данных также контролируется операторами сервиса.

Проведённый анализ существующих систем показывает, что большинство современных платформ онлайн-голосования ориентированы на удобство использования и быстроту проведения опросов, однако не всегда обеспечивают достаточный уровень

прозрачности и проверяемости результатов. Централизованное хранение данных создаёт потенциальные риски изменения информации, а также снижает уровень доверия пользователей к системе.

В связи с этим возникает необходимость разработки систем голосования, основанных на децентрализованных технологиях. Применение таких технологий позволяет обеспечить неизменяемость записей о голосовании, повысить прозрачность процедуры и предоставить пользователям возможность независимой проверки результатов.

1.3 Обзор технологий для реализации программного обеспечения

Современные *Web*-приложения разрабатываются с использованием разнообразных технологий и инструментов, которые позволяют создавать масштабируемые, надёжные и безопасные информационные системы. Для реализации системы онлайн-голосований, обеспечивающей прозрачность и возможность независимой проверки результатов, был выбран комплекс технологических решений, включающий клиентскую часть, серверную часть, систему управления базами данных и блокчейн-технологии.

Архитектура системы построена по принципу клиент-серверной модели: пользователь взаимодействует с интерфейсом веб-приложения через браузер, при этом клиентская часть отвечает за отображение интерфейса и обработку действий пользователя, а серверная часть обеспечивает обработку запросов, хранение данных и реализацию бизнес-логики. Взаимодействие между компонентами осуществляется через сетевые протоколы передачи данных и специализированные *API*-интерфейсы.

JavaScript является одним из ключевых языков программирования для разработки *Web*-приложений. Он применяется для создания интерактивных интерфейсов, обработки событий и динамического обновления содержимого страниц. Основными особенностями языка являются динамическая типизация, автоматическое управление памятью и поддержка асинхронного программирования. С помощью технологии *AJAX JavaScript* обеспечивает обмен данными между клиентом и сервером без полной перезагрузки страницы, что повышает скорость работы интерфейса и удобство взаимодействия с пользователем.

Для построения пользовательского интерфейса используется библиотека *React*, которая широко применяется в современных *Web*-приложениях. *React* поддерживает компонентный подход: интерфейс делится на отдельные компоненты, каждый из которых реализует определённую часть функциональности системы. Такой подход облегчает разработку, повышает повторное использование кода и упрощает поддержку программного обеспечения. Механизм *Virtual DOM*, используемый в *React*, позволяет оптимизировать обновление интерфейса, изменяя только те элементы, которые действительно были изменены, что повышает производительность приложения.

Серверная часть разрабатывается на платформе *ASP.NET*, входящей в экосистему *Microsoft .NET*, и предназначенной для создания Web-приложений, *API*-сервисов и распределённых систем. Основным языком разработки является C#, современный объектно-ориентированный язык со строгой типизацией. *ASP.NET* предоставляет

инструменты для обработки *HTTP*-запросов, управления пользовательскими данными, реализации механизмов безопасности и интеграции с базами данных.

Серверная часть реализует *REST API* (*Representational State Transfer Application Programming Interface*), обеспечивающее взаимодействие между клиентской и серверной частями приложения. В рамках *REST API* используются стандартные *HTTP*-методы: *GET* – получение данных; *POST* – создание новых записей; *PUT* – обновление существующих данных; *DELETE* – удаление данных. Передача данных между клиентом и сервером осуществляется в формате *JSON*, что обеспечивает гибкость архитектуры и возможность масштабирования системы.

Для хранения данных используется система управления базами данных *PostgreSQL*, которая обеспечивает надёжное хранение информации, поддержку транзакций и возможность выполнения сложных запросов. В базе данных хранятся сведения о пользователях, голосованиях, кандидатах или вариантах выбора, результатах голосований и другие необходимые данные системы.

Для обеспечения безопасного доступа к системе применяется механизм *JWT* (*JSON Web Token*). После успешной авторизации сервер формирует токен, который содержит зашифрованную информацию о пользователе и его правах доступа. *JWT* состоит из трёх частей: *Header* (заголовок), *Payload* (полезная нагрузка) и *Signature* (подпись).

– *header* содержит информацию о типе токена и используемом алгоритме подписи;

– *payload* включает данные о пользователе и его правах доступа, а также дополнительные сведения, например время жизни токена;

– *signature* формируется на основе заголовка и полезной нагрузки с использованием секретного ключа или пары ключей (в случае асимметричной криптографии), что обеспечивает целостность и защиту от подделки токена.

Токен сохраняется на стороне клиента и передаётся серверу при выполнении последующих запросов. Сервер проверяет подпись и расшифровывает *payload* для определения прав доступа пользователя, что позволяет безопасно управлять доступом к ресурсам системы без необходимости хранения сессий на сервере.

Для обеспечения прозрачности и проверяемости голосований используется технология блокчейн на базе сети *Ethereum*. *Ethereum* представляет собой децентрализованную платформу для создания и выполнения смарт-контрактов. Информация о голосах хранится в распределённом реестре блокчейн-сети, который состоит из последовательности блоков, связанных криптографически. После добавления данных в блокчейн изменить или удалить их невозможно, что обеспечивает высокий уровень безопасности и доверие пользователей к результатам голосований.

Solidity используется для разработки смарт-контрактов, которые автоматически выполняются при наступлении заданных условий. Смарт-контракты реализуют регистрацию участников голосования, фиксацию голосов, защиту от повторного голосования и автоматический подсчёт результатов. После публикации смарт-контракта его код становится неизменяемым, что гарантирует соблюдение правил работы системы.

Для взаимодействия пользователей с блокчейн-сетью используется криптокошелёк *MetaMask*, позволяющий безопасно подписывать транзакции и фиксировать голоса в блокчейне. Клиентская часть приложения подключается к

кошельку и может выполнять операции: подключение к кошельку, отправка транзакций голосования, получение информации о выполненных действиях и чтение данных из смарт-контрактов.

Использование библиотек *Web3.js* или *Ethers.js* обеспечивает удобную интеграцию *Web*-приложения с блокчейн-инфраструктурой и позволяет клиентской части напрямую взаимодействовать со смарт-контрактами.

Таким образом, комплексное применение современных технологий, включая клиентскую и серверную части, базы данных, блокчейн и смарт-контракты, обеспечивает разработку системы онлайн-голосований, обладающей высокой степенью безопасности, прозрачности, проверяемости и удобства для всех участников процесса.

1.4 Анализ инструментальных средств автоматизации разработки и тестирования

Для разработки программного комплекса системы онлайн-голосований будет использован комплекс современных инструментов и технологий, обеспечивающих удобство разработки, безопасное взаимодействие с блокчейном и управление данными. Применение специализированных средств повышает эффективность работы, облегчает управление проектом и обеспечивает более высокое качество программного продукта.

В качестве основной среды разработки выбран *JetBrains Rider*, а для работы с контейнеризацией и развёртыванием сервисов используется *Docker*. Хранение и работа с исходным кодом осуществляется через *Git* и платформу *GitHub*. Для управления реляционными базами данных применяется *DataGrip*, а тестирование *API* выполняется с помощью *Postman*. Взаимодействие с блокчейн-сетью реализовано через криптокошелёк *MetaMask* и инструменты разработки смарт-контрактов *Hardhat* с сетью тестирования *Sepolia*. Для проектирования и визуализации архитектуры системы используется *StarUML*.

JetBrains Rider представляет собой современную интегрированную среду разработки (*IDE*) для создания приложений на платформе *.NET*, разработанную компанией *JetBrains*. Она предоставляет широкий набор инструментов для программирования на *C#,* включая разработку серверной части *Web*-приложений на *ASP.NET*, реализацию *REST API*, работу с базами данных и бизнес-логикой. Основные возможности *Rider* включают интеллектуальное автодополнение кода, встроенную отладку, анализ и поиск ошибок, интеграцию с *Git*, поддержку *Docker* и мощные инструменты рефакторинга. Высокая производительность и удобный интерфейс позволяют эффективно работать с крупными проектами, а наличие множества плагинов расширяет функциональность *IDE*.

Docker является платформой для контейнеризации, обеспечивающей запуск приложений в изолированных контейнерах со всеми необходимыми зависимостями. Это упрощает развертывание и тестирование системы онлайн-голосований, позволяя контейнерам работать одинаково на разных серверах и операционных системах. Применение *Docker* обеспечивает изоляцию приложений, упрощает масштабирование, повышает стабильность работы и облегчает процесс деплоя. В проекте *Docker*

используется для контейнеризации серверной части и базы данных, гарантируя стабильную работу приложения.

Для управления версиями исходного кода в процессе разработки используется распределённая система контроля версий *Git*. Данный инструмент позволяет отслеживать изменения в программном коде, фиксировать этапы разработки, а также при необходимости возвращаться к предыдущим версиям проекта. Применение *Git* обеспечивает удобную работу с ветками разработки, что позволяет параллельно реализовывать различные функциональные возможности системы и впоследствии объединять внесённые изменения.

Для удалённого хранения репозитория и организации совместной работы над проектом используется платформа *GitHub*, которая тесно интегрируется с системой контроля версий *Git*. Платформа предоставляет инструменты для размещения исходного кода в удалённом репозитории, управления ветками, отслеживания изменений и взаимодействия между разработчиками. Использование *GitHub* обеспечивает безопасное хранение программного кода, упрощает процесс коллективной разработки и способствует эффективному управлению жизненным циклом проекта.

DataGrip, разработанный компанией *JetBrains*, предоставляет графический интерфейс для работы с базами данных и выполнения *SQL*-запросов. Он поддерживает различные СУБД, включая *PostgreSQL*, применяемую в данном проекте. Основные возможности *DataGrip* включают выполнение запросов, визуальное управление таблицами, анализ структуры базы данных, автодополнение *SQL*-кода и редактирование данных, что облегчает разработку и администрирование базы данных приложения.

Postman позволяет тестировать *REST API*, отправляя *HTTP*-запросы к серверу и анализируя ответы. Он поддерживает различные методы запросов (*GET*, *POST*, *PUT*, *DELETE*), передачу данных в формате *JSON*, проверку авторизации и аутентификации, создание коллекций запросов и автоматизацию тестирования. Использование *Postman* ускоряет процесс разработки серверной части приложения.

MetaMask выступает криптовалютным кошельком для взаимодействия пользователей с блокчейн-сетью *Ethereum*. Он используется для подписания транзакций голосов в сети *Ethereum*, позволяя безопасно работать с блокчейн-инфраструктурой без хранения приватных ключей на сервере.

Hardhat – это инструмент для разработки, тестирования и развертывания смарт-контрактов в сети *Ethereum*. Он позволяет компилировать контракты на *Solidity*, запускать локальную тестовую сеть *Ethereum*, автоматизировать тестирование и деплой контрактов в тестовые и публичные сети. Одним из ключевых преимуществ *Hardhat* является возможность локального тестирования смарт-контрактов без совершения реальных транзакций, что существенно упрощает процесс разработки блокчейн-компонентов системы онлайн-голосований.

StarUML – это приложение для создания *UML*-диаграмм, которое позволяет разработчикам создавать модели проектов, планировать архитектуру и дизайн приложений. *StarUML* имеет графический интерфейс пользователя, который предоставляет множество инструментов для создания и редактирования диаграмм, включая диаграммы классов, диаграммы последовательностей, диаграммы состояний, диаграммы активностей и многое другое. На рисунке 1.3 в виде выпадающего меню представлен весь список возможных диаграмм.

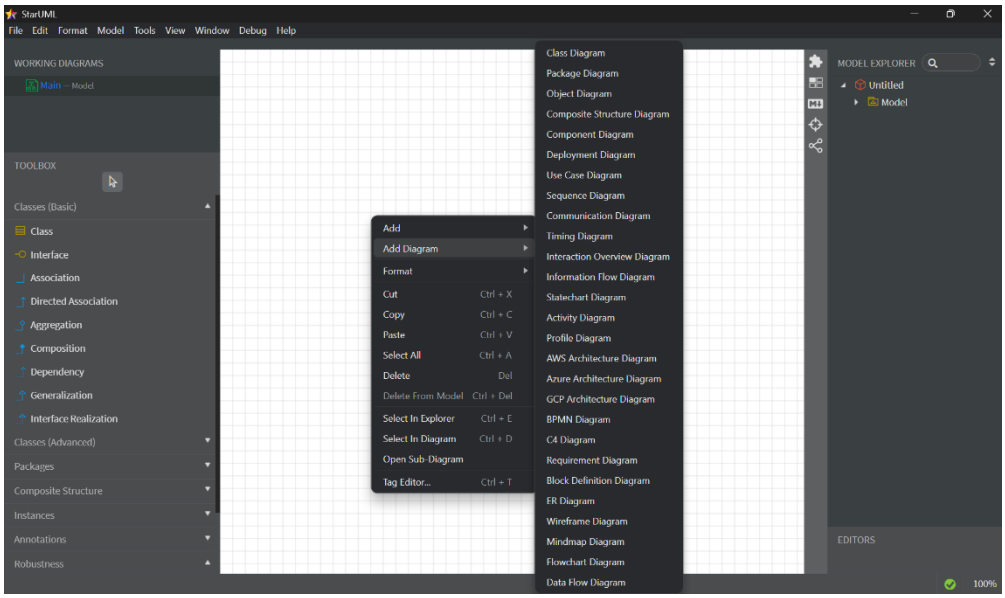

Рисунок 1.3 – Интерфейс главного окна *StarUml*

В *StarUML* можно создавать различные типы диаграмм, включая:

– диаграммы классов – используются для описания структуры классов и их отношений;

– диаграммы последовательностей – используются для описания взаимо действия между объектами и процессов, происходящих во времени;

– диаграммы состояний – используются для описания жизненного цикла объекта и его состояний;

– диаграммы компонентов – используются для описания структуры и отношений между компонентами системы;

– диаграммы развертывания – используются для описания физического размещения компонентов и системы в целом;

– диаграммы активностей – используются для описания последовательности действий и процессов в системе.

Данное приложение имеет множество функций, которые облегчают создание и редактирование диаграмм, такие как автоматическое выравнивание элементов, возможность группировки элементов, подсветка синтаксиса, подсказки и многое другое. Кроме того, *StarUML* позволяет пользователю создавать свои собственные элементы, шаблоны и плагины, чтобы улучшить функциональность и адаптировать инструмент под свои потребности.

1.5 Постановка задачи на дипломное проектирование

Цель разработки: разработать веб-приложение, предназначенное для автоматизации процесса проведения онлайн-голосований в системе образования.

Данный программный комплекс предназначается для использования в образовательных учреждениях для организации голосований, опросов и принятия

коллективных решений среди студентов, преподавателей и администрации. Особенностью разрабатываемой системы является использование технологии блокчейн, обеспечивающей прозрачность, неизменяемость и надежность хранения данных о голосовании.

Для администраторов система предусматривает следующие функции:

– управление пользователями системы;

– назначение ролей пользователям;

– создание и управление голосованиями;

– модерация контента и голосований;

– просмотр статистики и результатов голосований;

– управление новостями системы.

Для преподавателей или организаторов голосований приложение предусматривает следующие функции:

– создание голосований и опросов;

– настройка параметров голосования (дата начала и окончания, варианты ответов, тип голосования);

– редактирование и управление созданными голосованиями;

– просмотр результатов голосований и статистики участия;

– публикация информационных материалов и новостей.

Для участников голосования приложение предусматривает следующие функции:

– просмотр списка доступных голосований;

– ознакомление с описанием голосований;

– участие в голосовании с выбором одного или нескольких вариантов ответа;

– просмотр результатов завершённых голосований;

– получение уведомлений о новых голосованиях и публикации результатов.

Приложение должно иметь следующую структуру и функциональность:

– работать как веб-приложение, основанное на архитектуре REST;

– обеспечивать разграничение прав доступа пользователей в зависимости от их роли;

– предоставлять удобный пользовательский интерфейс для создания и проведения голосований;

– обеспечивать автоматический подсчёт голосов и формирование статистики;

– обеспечивать защиту от повторного голосования и несанкционированного доступа;

– предусматривать использование технологии блокчейн для фиксации результатов голосования, что позволяет обеспечить неизменяемость данных и возможность проверки подлинности голосов;

– быть кроссплатформенным и доступным через современные веб-браузеры;

– обеспечивать хранение и обработку данных пользователей и результатов голосований.

В результате разработки должно быть создано веб-приложение, позволяющее организовывать и проводить голосования в образовательной среде, обеспечивая удобство использования, прозрачность процесса голосования и автоматизированную обработку результатов.

2 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Общая структура приложения

2.2 Функциональная модель программного комплекса

2.3 Информационная модель программного комплекса

1. Таблица roles

Поле	Описание	Тип	Обязательное	Уникальное
id	Идентификатор роли	integer	да	да
name	Название роли (user, organizer, admin)	varchar(50)	да	да

---

2. Таблица users

Поле	Описание	Тип	Обязательное	Уникальное
id	Идентификатор пользователя	uuid	да	да
wallet_address	Адрес кошелька пользователя	varchar(255)	да	да
email	Email пользователя	varchar(255)	нет	да
role_id	Роль пользователя	integer	да	нет
group_id	Ссылка на группу пользователя	uuid	нет	нет
created_at	Дата регистрации	timestamp	да	нет

---

3. Таблица user_groups

Поле	Описание	Тип	Обязательное	Уникальное
id	Идентификатор группы	uuid	да	да
name	Название группы (например, факультет/курс)	varchar(255)	да	да

---

4. Таблица voting_types

Поле	Описание	Тип	Обязательное	Уникальное
id	Идентификатор типа голосования	integer	да	да
name	Тип голосования (single, multiple, ranking, open_answer)	varchar(50)	да	да
description	Описание типа	text	нет	нет

---

5. Таблица votings

Поле	Описание	Тип	Обязательное	Уникальное
id	Идентификатор голосования	uuid	да	да
title	Название голосования	varchar(255)	да	нет
description	Описание голосования	text	нет	нет
created_by	Организатор голосования	uuid	да	нет
type_id	Тип голосования	integer	да	нет
anonymous	Анонимное голосование	boolean	да	нет
start_time	Время начала	timestamp	да	нет
end_time	Время окончания	timestamp	да	нет
status	Статус (draft, active, finished)	varchar(50)	да	нет
created_at	Дата создания	timestamp	да	нет

---

6. Таблица candidates

Поле	Описание	Тип	Обязательное	Уникальное
id	Идентификатор кандидата	uuid	да	да
voting_id	Голосование	uuid	да	нет
user_id	Ссылка на пользователя, если кандидат человек	uuid	нет	нет
type	Тип кандидата (user, project, report)	varchar(50)	да	нет
name	Имя кандидата / название объекта	varchar(255)	да	нет
description	Описание кандидата / проекта	text	нет	нет

---

7. Таблица votes

Поле	Описание	Тип	Обязательное	Уникальное
id	Идентификатор голоса	uuid	да	да
voting_id	Голосование	uuid	да	нет
candidate_id	Выбранный кандидат	uuid	да	нет
user_id	Пользователь, который голосует	uuid	да	нет
vote_hash	Хэш голоса	varchar(255)	да	да
created_at	Время голосования	timestamp	да	нет

---

8. Таблица blockchain_records

Поле	Описание	Тип	Обязательное	Уникальное
id	Идентификатор записи	uuid	да	да
voting_id	Голосование	uuid	да	да
transaction_hash	Хэш транзакции в блокчейне	varchar(255)	да	да
votes_hash	Хэш всех голосов	varchar(255)	да	нет

Поле	Описание	Тип	Обязательное	Уникальное
block_number	Номер блока	integer	нет	нет
created_at	Время записи в блокчейн	timestamp	да	нет

---

9. Таблица audit_log

Поле	Описание	Тип	Обязательное	Уникальное
id	Идентификатор записи	uuid	да	да
user_id	Пользователь	uuid	да	нет
action_type	Тип действия (login, vote, create_voting, edit_voting и т.д.)	varchar(100)	да	нет
entity_id	Ссылка на объект действия (voting, candidate, news)	uuid	нет	нет
entity_type	Тип объекта (voting, candidate, news)	varchar(50)	нет	нет
details	Дополнительная информация (json)	json	нет	нет
timestamp	Время действия	timestamp	да	нет

---

10. Таблица notifications

Поле	Описание	Тип	Обязательное	Уникальное
id	Идентификатор уведомления	uuid	да	да
user_id	Пользователь	uuid	да	нет
type	Тип уведомления (email, system)	varchar(50)	да	нет
title	Заголовок уведомления	varchar(255)	да	нет
message	Текст уведомления	text	да	нет
read	Прочитано	boolean	да	нет
created_at	Время создания	timestamp	да	нет

---

11. Таблица news

Поле	Описание	Тип	Обязательное	Уникальное
id	Идентификатор новости	uuid	да	да
title	Заголовок	varchar(255)	да	нет
body	Текст новости	text	да	нет
author_id	Автор	uuid	да	нет
published_at	Дата публикации	timestamp	да	нет
pinned	Закрепить новость	boolean	да	нет
attachments	Файлы / вложения	json	нет	нет